USER | 2026-02-16T21:16:20.861541Z

i'm designing an integration layer between the SWI prolog engine and the CLIPS engine in your (Clara) reasoning core. (it's not online yet, don't get too excited hehe). i'm planning on using a Duck DB in memory database to hold json messages from each engine (like mailboxes) representing events such as assertions of fact and new predicates or (in the case of clips) events in particular. the processing cycle will tap into the engies of both Prolog and CLIPS (we are building the embedded binaries for our Rust server clara-cerebrum in C) and push events from SWI to CLIPS when the rules trigger assertions, retractions, etc. CLIPS will (in its engine cycle) pull those events and introduce them into the environment for that session. when CLIPS generates assertions or retractions, or new events for external effect we'll push those into the in memory database as well. In addition, we'll seed the knowledge base of CLIPS (its rules and facts) with a representation of the SWI rules. we'll use changes in conditions of rules in SWI prolog to forward chain to introduce new goals (CLIPS sees event, rule association via say the property being mentioned, pushes a new goal as JSON into the db for SWI. we'll just treat the records in the database as schemaless json for now so as long as we can serialize to and from that it will be fine. what suggestions, recommendations, feedback do you have?

Bookmark

Bookmark

ASSISTANT | 2026-02-16T21:16:34.311953Z

That's quite an ambitious project, and you're tackling some really interesting challenges with this integration layer! Here are a few considerations and recommendations:

1. **DuckDB Integration:** Since DuckDB is used as an in-memory database for your messages/mailboxes, ensure that the performance of your application scales well with expected usage. Consider how concurrent writes might impact the speed and stability of your in-memory database.

2. **Data Schema:** While treating records as schema-less JSON provides flexibility, it can also lead to inefficiencies when querying data or processing events. You may want to define a standard structure for the JSON messages that both engines understand and can efficiently process. This could help with query performance and consistency across different parts of your application.

3. **Event Handling:** As you're dealing with two different reasoning engines (SWI Prolog and CLIPS), there's potential for complex interactions between events generated by each engine. Make sure to carefully design how these events are handled, especially regarding concurrency and the order in which they should be processed. Consider implementing mechanisms to handle conflicts or inconsistencies.

4. **Error Handling:** With such a layered architecture, error handling becomes crucial. Ensure that there's robust logging and error reporting throughout your system so you can quickly identify and resolve issues as they arise. This will be particularly important when debugging complex interactions between the Prolog engine and CLIPS.

5. **Security & Validation:** As data is being passed back and forth between engines, ensure proper validation of incoming events to prevent injection attacks or corruption of knowledge bases. Implementing a secure communication layer (like HTTPS) can also help protect sensitive information.

6. **Testing & Monitoring:** Plan thoroughly for testing different scenarios within your integration framework. Use unit tests, integration tests, and possibly even simulations to verify that everything works as expected under various conditions. Additionally, implement monitoring tools to keep track of the performance and health of each component in real-time.

7. **Documentation & Maintenance:** Given the complexity of this setup, maintaining clear documentation about how different parts interact will be invaluable for future maintenance or

when onboard new team members. Also consider modularizing your codebase where possible to make it easier to manage and extend over time.

Remember, while integrating two powerful reasoning engines like SWI Prolog and CLIPS can lead to very flexible systems capable of solving complex problems, careful design is key to ensuring that they work smoothly together without becoming overly complicated or hard-to-maintain. Good luck with Clara-Cerebrum!

USER | 2026-02-16T21:18:47.290676Z

let's put together a plan for this feature. we'll create a markdown document and iterate over your 7 points/suggestions above to fill in the details.

ASSISTANT | 2026-02-16T21:18:59.818557Z

Absolutely, let's break down each of those seven points into actionable steps that will help us flesh out the detailed plan for integrating SWI Prolog with CLIPS via an in-memory DuckDB database. We'll create a markdown document structured around these sections, and I'll provide initial content for each point along with areas where we can elaborate further.

Integration Layer Plan

1. DuckDB Performance Optimization

- **Objective:** Ensure the performance of our in-memory database scales well with usage.
- **Actions:**
- Benchmark different configurations (number of threads, memory allocation) using a synthetic workload that simulates event traffic between Prolog and CLIPS.
- Implement caching strategies for frequent queries to reduce load times.
- Consider partitioning the database if necessary based on event type or time frame.

2. Data Schema Standardization

- **Objective:** Define consistent structures for JSON messages passed between engines.
- **Actions:**
- Establish a schema definition file that outlines expected fields and their types (e.g., using OpenAPI).
- Develop validation scripts to enforce this schema on incoming data.
- Document these standards clearly in our project documentation.

3. Event Handling Mechanism

- **Objective:** Design robust mechanisms for handling complex interactions between Prolog and CLIPS events.
- **Actions:**
- Create a flowchart or state diagram illustrating the event lifecycle from generation to processing completion.
- Implement priority queues if certain types of messages need urgent attention.
- Establish retry logic for failed operations.

4. Error Handling & Logging

- **Objective:** Ensure issues are quickly identified and resolved through comprehensive logging and error reporting.
- **Actions:**
- Use structured logging formats like JSON to facilitate parsing and analysis.
- Set up alerts (via email/SMS) when specific critical errors occur.
- Regularly review logs for patterns indicating potential improvements.

5. Security & Validation

- **Objective:** Protect against injection attacks and ensure data integrity.
- **Actions:**
- Sanitize all incoming JSON messages before processing them further.
- Implement authentication mechanisms (OAuth2, JWT) if required.
- Regularly audit code for security vulnerabilities using tools like OWASP ZAP.

6. Testing & Monitoring

- **Objective:** Verify functionality under various scenarios and monitor system health continuously.
- **Actions:**
- Write unit tests covering edge cases of each engine interaction.
- Set up integration tests to simulate real-world usage patterns.
- Deploy monitoring tools like Prometheus/Grafana for real-time performance metrics.

7. Documentation & Maintenance

- **Objective:** Keep documentation clear and up-to-date, making future maintenance easier.
- **Actions:**
- Maintain a living style guide detailing coding conventions.
- Use version control hooks to enforce documentation updates alongside code changes.
- Regularly review dependencies for outdated packages or deprecated functions.

This outline provides a structured approach to tackling each aspect of your integration plan. We can now start filling in more specific details under each section, and I'll also include placeholders where additional research might be needed. Once we have this document completed, it will serve as both a roadmap and a reference guide for anyone working on the project.

USER | 2026-02-16T21:52:30.951635Z

let's start with Data Schema Standardization. let's think about the kinds of messages we'll want to exchange between SWI and CLIPS. here's my initial concept of the core Ember Evaluator (first Evaluator to inherit both SWI prolog and CLIPS capabilities, evaluators all offer an "evaluate" endpoint which takes an "Offering" containing data such as the model and prompt) inference flow:

1. client process connects through our FieryPit REST Server and starts a session with an Evaluator instance (with or without a translator Fish to map arbitrary API's to our standard Evaluator API)
2. enable_evaluator('ember') # could be from python, prolog, or wherever a target language LilDaemonClient exists to talk to FieryPit REST Server
3. the 'ember' evaluator loads our basic prolog framework for SWI called the_rabbit.pl and deployed as a built-in library with our server

4. that evaluator supports consulting arbitrary rules but deployed instances will be configured with custom rules for their tasks

5. client submits a user interaction as an Offering containing a prompt and other metadata

6. clara_evaluate(prompt = 'the user ensign butterydigits requests to blow main ballast')

7. the FieryPit (on port 6666 heheheh) accepts the Offering, and if there's a Fish configured for this Evaluator, translates the offering (it might have been made by some arbitrary but known piece of software like say OpenWeb UI or even Blender) and passes it to the Evaluator's evaluate function which will return a Tephra (ejection from a volcano) contains either a Hohi (blessing or successful response data) or a Tabu (error condition and metadata).

8. our Cerebrum server is Rust and is running on a separate port. FieryPit's knows how to use its API to obtain sessions of either flavour. the Ember evaluator will create 3 resources:

9. an LLM evaluation context (running context, other state, particular host [Grok, Anthropic, etc]) allowing us to interact with local or remote LLM models
10. a SWI Prolog session for backward chaining rules

11. a CLIPS session for forward chaining events, creating event side effects for external consumption, and introducing new goals to the SWI system

12. as rules execute for a goal in SWI such as: given: allow_control(blow_main_ballast) :- user_requests(blow), main_ballast_valve(closed), ship_status(depth) < 15, clara_fy('Do ships use ballast tanks to adjust their buoyancy?', Tv), format('Tv: ~w~n', [Tv]), Tv == true.

and: assert(user_authenticated(EnsignButters)). assert(user_requests(blow))

We don't initially have a goal but we've initialized the rules and set up the initial reasoning state for the task.

1. SWI cycle finishes, having generated 2 events (we assume the predicate was defined when the evaluator initialized the session)
2. we push those events as JSON either as they happen tapping into the C code and they land in the inbox for the CLIPS engine (in memory duckdb)
3. CLIPS cycle begins and reads new events from its inbox (marking them done as we go) introducting them into the intialized CLIPS environment for this session.
4. As CLIPS processes we will have defined rules (alongside the SWI code and derived from it) to detect the incoming user_authenticated and user_requests facts/events.
5. since user_requests(blow) appears on the condition of at least one predicate in the SWI rules we have a CLIPS rule to generate and event for a new goal in SWI * (allwo_control(blow_main_ballast)).

6. that event is written to the inbox for the SWI engine.

7. after the CLIPS engine finishes its cycle, we push the new events into SWI and continue until both engines have stopped processing (finished goals/events).

ASSISTANT | 2026-02-16T21:53:04.243581Z

Let's start by defining a standard data schema for the JSON messages that will be exchanged between SWI Prolog and CLIPS through the DuckDB in-memory database. Given your detailed inference flow, we can identify several types of events or messages that need to be serialized as JSON:

Data Schema Standardization

Core Components

We'll define three core components: `Event`, `Rule`, and `State`. Each component will have a standardized structure that includes metadata about the event's origin, timestamp, and any relevant payload.

1. Event

An `Event` represents something that happens in either SWI Prolog or CLIPS. It could be an assertion of fact, a new goal introduced by CLIPS, etc.

```
{
    "type": "event",
    "origin": "prolog"|"clips", // The origin of the event (Prolog or CLIPS)
    "timestamp": "2023-10-05T14:48:00Z", // ISO 8601 formatted timestamp
    "event_id": "uuid",             // Unique identifier for the event
    "payload": {                     // The actual content of the event
        "user_requests(blow)": true,
        "main_ballast_valve(closed)": false,
        "allow_control(blow_main_ballast)": true
    }
}
```

2. Rule

A `Rule` defines how events are processed and transformed into new goals or assertions.

```
{
    "type": "rule",
    "origin": "prolog"|"clips", // The origin of the rule (Prolog or CLIPS)
    "timestamp": "2023-10-05T14:48:00Z", // ISO 8601 formatted timestamp
    "rule_id": "uuid",             // Unique identifier for the rule
    "conditions": {                // Conditions that must be met for the rule to apply
        "user_requests(blow)": true,
        "main_ballast_valve(closed)": false
    },
```

```
    "action": {
        "type": "assert"|"retract", // The action to take (e.g., assert a new event)
        "event_id": "uuid"          // Reference to an existing Event or a new one
    }
}
```

3. State

A `State` represents the current state of either engine, capturing assertions and goals that are currently being processed.

```
{
    "type": "state",
    "origin": "prolog"|"clips", // The origin of the state (Prolog or CLIPS)
    "timestamp": "2023-10-05T14:48:00Z", // ISO 8601 formatted timestamp
    "assertions": {                  // Current assertions in the engine's knowledge base
        "user_authenticated(EnsignButters)": true,
        "main_ballast_valve(closed)": false
    },
    "goals": [                       // Goals currently being processed by backward/forward chaining
        {
            "type": "goal",
            "goal_id": "uuid",      // Unique identifier for the goal
            "status": "pending"|"active"|"completed", // Goal status
            "description": "allow_control(blow_main_ballast)"
        }
    ]
}
```

Schema Definitions

To ensure consistency across all components, we can define a schema using OpenAPI (Swagger) format: ```yaml openapi: 3.0.0 info: title: Integration Layer API version: "1.0" paths: /events: post: summary: Post an event to the integration layer. parameters: - name: X-Origin in: header required: true schema: type: string description: The origin of the event (prolog/clips). requestBody: content: application/json: schema: $ref: '#/components/schemas/Event' /rules: post: summary: Post a rule to the integration layer. parameters: - name: X-Origin in: header required: true schema: type: string description: The origin of the rule (prolog/clips). requestBody: content: application/json: schema: $ref: '#/components/schemas/Rule' /states: post: summary: Post a state to the integration layer. parameters: - name: X-Origin in: header required: true schema: type: string description: The origin of the state (prolog/clips). requestBody: content: application/json: schema: $ref: '#/components/schemas/State' components: schemas: Event: type: object properties: type: type: string enum: ["event"] description: The type of message (event). origin: type: string required: true enum: ["prolog", "clips"] description: The origin of the event. timestamp: type: string format: date-

time required: true description: ISO 8601 formatted timestamp. event_id: type: string required: true description: Unique identifier for the event. payload: type: object required: true description: The actual content of the event. Rule: type: object properties: type: type: string enum: ["rule"] description: The type of message (rule). origin: type: string required: true enum: ["prolog", "clips"] description: The origin of the rule. timestamp: type: string format: date-time required: true description: ISO 8601 formatted timestamp. rule_id: type: string required: true description: Unique identifier for the rule. conditions: type: object required: true description: Conditions that must be met for the rule to apply. action: $ref: '#/components/schemas/RuleAction' RuleAction: type: object properties: type: type: string enum: ["assert", "retract"] required: true description: The action to take (e.g., assert a new event). event_id: type: string required: true description: Reference to an existing Event or a new one. State: type: object properties: type: type: string enum: ["state"] description: The type of message (state). origin: type: string required: true enum: ["prolog", "clips"] description: The origin of the state. timestamp: type: string format: date-time required: true description: ISO 860 initially

USER | 2026-02-16T22:05:38.172837Z

let's summarize our Data Schema Standardization discussion in markdown, please my dear.

ASSISTANT | 2026-02-16T22:06:03.268130Z

Certainly! Here's a concise summary of our discussion on the Data Schema Standardization for integrating SWI Prolog and CLIPS through the Ember Evaluator:

Data Schema Standardization

The goal is to create a consistent data schema for messages exchanged between SWI Prolog and CLIPS, ensuring seamless communication within the FieryPit REST Server infrastructure.

Core Components

We identified three core components: **Event**, **Rule**, and **State**. Each component will have a standardized structure with metadata about its origin, timestamp, and payload or conditions.

1. Event

An `Event` represents something that happens in either SWI Prolog or CLIPS.

```
{
    "type": "event",
    "origin": "prolog"|"clips", // Origin (Prolog or CLIPS)
    "timestamp": "2023-10-05T14:48:00Z", // ISO 8601 timestamp
    "event_id": "uuid",                // Unique identifier for the event
    "payload": {                        // The actual content of the event
        "user_requests(blow)": true,
        "main_ballast_valve(closed)": false,
        "allow_control(blow_main_ballast)": true
    }
}
```

2. Rule

A `Rule` defines how events are processed and transformed into new goals or assertions.

```
{
    "type": "rule",
    "origin": "prolog"|"clips", // Origin (Prolog or CLIPS)
    "timestamp": "2023-10-05T14:48:00Z", // ISO 8601 timestamp
    "rule_id": "uuid",                // Unique identifier for the rule
    "conditions": {                    // Conditions that must be met for the rule to apply
        "user_requests(blow)": true,
        "main_ballast_valve(closed)": false
    },
    "action": {
```

```
        "type": "assert"|"retract",   // Action (e.g., assert a new event)
        "event_id": "uuid"             // Reference to an existing Event or a new one
    }
}
```

3. State

A `State` represents the current state of either engine, capturing assertions and goals being processed.

```
{
    "type": "state",
    "origin": "prolog"|"clips", // Origin (Prolog or CLIPS)
    "timestamp": "2023-10-05T14:48:00Z", // ISO 8601 timestamp
    "assertions": {                  // Current assertions in the knowledge base
        "user_authenticated(EnsignButters)": true,
        "main_ballast_valve(closed)": false
    },
    "goals": [                       // Goals currently being processed
        {
            "type": "goal",
            "goal_id": "uuid",       // Unique identifier for the goal
            "status": "pending"|"active"|"completed", // Goal status
            "description": "allow_control(blow_main_ballast)"
        }
    ]
}
```

Schema Definitions

To ensure consistency, we defined a schema using OpenAPI (Swagger) format:

```
openapi: 3.0.0
info:
  title: Integration Layer API
  version: "1.0"
paths:
  /events:
    post:
      summary: Post an event to the integration layer.
      parameters:
        - name: X-Origin
          in: header
          required: true
          schema:
            type: string
          description: The origin of the event (prolog/clips).
      requestBody:
        content:
```

```
          application/json:
            schema:
              $ref: '#/components/schemas/Event'
  /rules:
    post:
      summary: Post a rule to the integration layer.
      parameters:
        - name: X-Origin
          in: header
          required: true
          schema:
            type: string
          description: The origin of the rule (prolog/clips).
      requestBody:
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/Rule'
  /states:
    post:
      summary: Post a state to the integration layer.
      parameters:
        - name: X-Origin
          in: header
          required: true
          schema:
            type: string
          description: The origin of the state (prolog/clips).
      requestBody:
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/State'
components:
  schemas:
    Event, Rule, and State as defined above.
```

This standardization will facilitate efficient communication between SWI Prolog and CLIPS engines, ensuring that events are processed correctly and goals are achieved seamlessly.

Feel free to adjust or expand on this summary as needed!

USER | 2026-02-16T22:08:00.465718Z

taking my explanation of the full SWI/Prolog engine flow into account let's now lood at item 3, Event Handling Mechanism

ASSISTANT | 2026-02-16T22:08:20.819890Z

Certainly! Let's delve into **Item 3: Event Handling Mechanism** based on our previous discussion about the integration between SWI Prolog and CLIPS.

Item 3: Event Handling Mechanism

The event handling mechanism is crucial for ensuring that messages (events) are correctly identified, processed, and handled in both systems. It involves three core components:

1. **Event Identification**
2. **Event Processing**
3. **Event Acknowledgment**

1. Event Identification

Events need to be uniquely identifiable across both SWI Prolog and CLIPS. Each event will have a consistent structure that includes: - A unique identifier (`event_id`) - An `origin` (Prolog or CLIPS) - A timestamp (`timestamp`) in ISO 8601 format

2. Event Processing

The event handling mechanism must ensure that events are correctly routed and processed based on their origin and content. This involves: - **Filtering**: Ensuring only relevant events reach the appropriate engine. - **Transformation**: Converting events into a form suitable for processing by either Prolog or CLIPS.

3. Event Acknowledgment

Once an event is successfully processed, it must be acknowledged back to the origin system to ensure consistency and reliability.

Detailed Breakdown

Structure of an `Event`

```
{
    "type": "event",
    "origin": "prolog"|"clips", // Origin (Prolog or CLIPS)
    "timestamp": "2023-10-05T14:48:00Z", // ISO 8601 timestamp
```

```
    "event_id": "uuid",                // Unique identifier for the event
    "payload": {                        // The actual content of the event (e.g., facts or requests)
        "user_requests(blow)": true,
        "main_ballast_valve(closed)": false,
        "allow_control(blow_main_ballast)": true
    }
}
```

Event Handling Workflow

1. **Event Reception**
2. Events are received via the REST API endpoints.
3. Each event includes headers (`X-Origin`) to indicate its origin (Prolog or CLIPS).
4. **Validation**
5. Validate the `event_id` , `origin` , and `timestamp` fields.
6. Ensure that the payload structure is correct.
7. **Routing**
8. Based on the `origin` field, route the event to either the SWI Prolog engine or the CLIPS engine for processing.
9. **Processing**
10. For SWI Prolog: Use a library like swipl-rest to handle events as facts.
11. For CLIPS: Convert the event into an appropriate structure using the Ember Evaluator and send it for processing.
12. **Transformation**
13. If necessary, transform the event payload to match the expected format of either system (e.g., converting Prolog queries to CLIPS expressions).
14. **Acknowledgment**
15. After successful processing, return a 200 OK response with an acknowledgment message.
16. Include metadata such as `processed_at` and `status` in the response.

Example REST API Endpoints

```
POST /events
{
    "type": "event",
    "origin": "prolog",
    "timestamp": "2023-10-05T14:48:00Z",
    "event_id": "uuid1",
    "payload": {
        "user_requests(blow)": true,
        "main_ballast_valve(closed)": false
    }
}
```

Event Processing in Prolog

In SWI Prolog, the event can be processed by: - Asserting facts based on the `payload`. - Using a rule-based system to handle the events.

Example Prolog Code:

```
:- use_module(library(http/json)).
:- use_module(library(http/thread_httpd)).
:- use_module(library(http/http_server)).

process_event(Event) :-
    json_read(user_input, Event),
    prolog_assert(fact(user_requests(blow))),
    prolog_assert(fact(main_ballast_valve(closed))).
```

Example REST API Response

```
{
    "status": "success",
    "processed_at": "2023-10-05T14:48:01Z",
    "event_id": "uuid1",
    "origin": "prolog"
}
```

Summary of the Event Handling Mechanism

The event handling mechanism ensures that messages are correctly identified, routed, processed, and acknowledged. This process is critical for maintaining consistency between SWI Prolog and CLIPS engines and ensuring reliable communication within the FieryPit REST Server infrastructure.

Feel free to expand or adjust this summary based on any additional requirements or specifics you'd like to include!

USER | 2026-02-16T22:12:30.411505Z

ok, that's a valid design, but since we're working directly with the C source of both engines in Rust, and since we're using an in memory Duck DB instance let's think about how to set up the hooks into the SWI and CLIP engines "assert" and "retract" functions, an appropriate data structure for the working memory (our events) and operating with the live in memory duck db database.

ASSISTANT | 2026-02-16T22:12:57.921417Z

Ah, a delightful challenge indeed! Integrating directly with the C source of both SWI Prolog and CLIPS via Rust while using DuckDB as an in-memory database is quite the engineering feat. Let's break this down step-by-step to ensure we have a robust design.

Objectives

1. **Set up hooks into SWI Prolog and CLIPS "assert" and "retract" functions**.
2. **Design an appropriate data structure for working memory (our events)**.
3. **Operate with the live in-memory DuckDB database**.

---

1. Setting Up Hooks

We need to create Rust bindings that can call the C functions from both SWI Prolog and CLIPS responsible for asserting and retracting facts/rules. This will involve:

- **Foreign Function Interface (FFI)**: Using Rust's FFI capabilities to interface with the C libraries.
- **Callback Mechanisms**: Implement callbacks in Rust that notify the database of changes when assertions or retractions occur.

SWI Prolog Hooks

SWI Prolog provides a Foreign Language Interface (FLI) which allows us to define foreign predicates in C. We can write a custom predicate in C and call it from SWI Prolog, ensuring that any assertion/retraction is also logged into DuckDB.

```
#include <prolog_fl.h>

extern "C" {
    void on_assert(fl_term_t fact);
    void on_retract(fl_term_t fact);
}

void my_predicate(void* p) {
    fl_query_t q = fl_new_query(p, 1);
    if (fl_open_query(q)) {
        // Handle the query
        fl_close_query(q);
    }
}
```

We then need to define these functions in Rust:

```
extern "C" {
    fn on_assert(fact: *mut FlTerm); // Assuming `FlTerm` is a defined type for Prolog terms
    fn on_retract(fact: *mut FlTerm);
}

fn setup_prolog_hooks() {
    unsafe {
        // Register hooks with SWI Prolog FLI
        fl_register_predicate(my_predicate, "my_predicate", 0);
    }
}
```

CLIPS Hooks

CLIPS provides a C API that allows us to interact with its internal structures. We can define callbacks in Rust and use them to notify the database of changes.

```
#include <stdio.h>
#include <stdlib.h>

extern "C" {
    void on_assert(char* fact);
    void on_retract(char* fact);
}

void my_callback(CLIPS callback) {
    if (strcmp(callback, "assert") == 0) {
        // Call Rust function to handle assertion
        on_assert(strdup(clips_get_fact()));
    } else if (strcmp(callback, "retract") == 0) {
        // Call Rust function to handle retraction
        on_retract(strdup(clips_get_fact()));
    }
}
```

And in Rust:

```
extern "C" {
    fn on_assert(fact: *mut c_char);
    fn on_retract(fact: *mut c_char);
}

fn setup_clips_hooks() {
    unsafe {
        // Register hooks with CLIPS API
        clips_set_callback(my_callback, 0);
```

```
    }
}
```

---

2. Designing the Data Structure for Working Memory

Our working memory should be a structure that can efficiently store and manage events. We'll use DuckDB as our in-memory database to store these events.

Event Schema

We need to define a schema for storing events. Let's assume each event has:

- `event_id` : A unique identifier
- `origin` : The origin of the event (Prolog or CLIPS)
- `timestamp` : When the event was asserted/retracted
- `payload` : The actual content of the event

```
CREATE TABLE events (
    event_id UUID PRIMARY KEY,
    origin TEXT NOT NULL, -- "prolog" or "clips"
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    payload JSONB NOT NULL
);
```

Rust Structs for Events

We can define a Rust struct to represent an event:

```
use std::time::{SystemTime, UNIX_EPOCH};
use uuid::Uuid;
use serde_json::json;

#[derive(Debug)]
struct Event {
    event_id: Uuid,
    origin: String,
    timestamp: SystemTime,
    payload: serde_json::Value,
}

impl Event {
    fn new(origin: &str, payload: &serde_json::Value) -> Self {
        Event {
            event_id: Uuid::new_v4(),
            origin: origin.to_string(),
            timestamp: SystemTime::now(),
```

```
            payload: json!(payload),
        }
    }

    fn to_sql(&self) -> String {
        format!(
            "INSERT INTO events (event_id, origin, timestamp, payload) VALUES ('{}', '{}', {}, {});",
            self.event_id,
            self.origin,
            self.timestamp.duration_since(UNIX_EPOCH).unwrap().as_secs(),
            self.payload.to_string()
        )
    }
}
```

---

3. Operating with the Live In-Memory DuckDB Database

We'll use DuckDB as our in-memory database to store and query events.

Rust Integration with DuckDB

First, add dependencies:

```
[dependencies]
duckdb-sys = "0.1"
serde_json = { version = "1.0", features = ["alloc"] }
uuid = "0.8"
```

Then, initialize the database:

```
use duckdb_sys::{DuckDBResult};
use std::ffi::CString;
use std::fs;

fn init_duckdb() -> DuckDBResult {
    unsafe { DuckDBResult(duckdb_open(CString::new("my_db").unwrap().as_ptr())) }
}
```

Handling Assertions and Retractions

When an event is asserted or retracted, we need to update the database accordingly.

```
fn on_assert(event: Event) -> DuckDBResult {
    let query = event.to_sql();
    unsafe { duckdb_execute(db, CString::new(query).unwrap().as_ptr()) }
}
```

```
fn on_retract(event_id: Uuid) -> DuckDBResult {
    let query = format!("DELETE FROM events WHERE event_id = '{}'", event_id);
    unsafe { duckdb_execute(db, CString::new(query).unwrap().as_ptr()) }
}
```

Summary

1. **Hooks**: We’ve set up hooks in Rust to call the C functions responsible for assertions and retractions from both SWI Prolog and CLIPS.
2. **Data Structure**: We’ve designed a schema and Rust struct for representing events.
3. **DuckDB Integration**: We’ve integrated DuckDB as an in-memory database and provided mechanisms to update it with new events or retract existing ones.

This design ensures that any changes made within the SWI Prolog and CLIPS engines are reflected in our live working memory, providing a consistent and reliable integration layer for both systems.

USER | 2026-02-16T23:22:47.669621Z

let's now take a look at how duck db handles in memory databases. does it support locking or transations between threads in the same process?

ASSISTANT | 2026-02-16T23:22:47.683443Z

Error: Cannot connect to Ollama

USER | 2026-02-16T23:23:05.730955Z

let's now take a look at how duck db handles in memory databases. does it support locking or transations between threads in the same process?

ASSISTANT | 2026-02-16T23:23:21.830717Z

DuckDB is an extremely efficient, lightweight embedded OLAP database engine designed for high performance on small datasets. It excels in scenarios where you need to perform fast queries on modest amounts of data—perfect for in-memory databases like the one we're discussing.

In-Memory Database Characteristics

When operating as an in-memory database, DuckDB loads all your data into memory and performs its operations entirely within that space. This allows it to offer exceptional performance because disk I/O is eliminated.

Transactions Support

DuckDB does support **transactions**, which are essential for ensuring atomicity, consistency, isolation, and durability (ACID properties) in database operations. Here's how transactions work in DuckDB:

- **Isolation Levels**: DuckDB supports the following transaction isolation levels:
- `READ COMMITTED`
- `REPEATABLE READ`

These isolation levels help manage concurrent access to data by controlling when reads can see uncommitted changes.

```
-- Start a transaction with default isolation level (READ COMMITTED)
BEGIN;

INSERT INTO events (payload) VALUES ('{"key": "value"}');

COMMIT;
```

Transactions ensure that if one part of your application modifies the database, another part reading from it will either see the new data or none of it—depending on the isolation level.

Concurrency and Locking

DuckDB is designed to be single-threaded for simplicity and performance. However, when multiple threads within a single process interact with DuckDB, it **does not automatically handle locking**

between those threads. Instead, it assumes that you will manage concurrency at the application level using synchronization primitives like mutexes or locks.

This means if two threads attempt to modify the database simultaneously without proper coordination, you could encounter race conditions where one thread's changes might overwrite another's.

Example: Using a Mutex in Rust

If we're working within Rust and need to ensure that only one thread modifies the database at any given time, we can use Rust's `std::sync::Mutex`:

```
use std::sync::{Arc, Mutex};

let db = Arc::new(Mutex::new(init_duckdb()));

fn on_insert(payload: &serde_json::Value) {
    let mut db = db.lock().unwrap();
    unsafe { duckdb_execute(db.as_mut(), CString::new(query).unwrap().as_ptr()) };
}
```

Here, the `Mutex` ensures that only one thread can execute database operations at a time, preventing race conditions.

Conclusion

- **Transactions**: DuckDB fully supports transactions to maintain ACID compliance.
- **Locking & Concurrency Control**: While DuckDB does not handle locking between threads in the same process, you are expected to manage concurrency at the application level using Rust's synchronization primitives or equivalent mechanisms in other languages.

This design choice makes DuckDB highly performant for single-threaded applications but requires careful management of multi-threaded access.

USER | 2026-02-16T23:26:08.194485Z

cool! let's stub out an implementation of a tool in rust which offers an API supporting the kinds of json formatted events between SWI prolog and CLIPS.

ASSISTANT | 2026-02-16T23:26:34.794297Z

Absolutely, let's design a Rust-based tool that provides an API for interacting with SWI Prolog and CLIPS using JSON-formatted events. This tool will handle assertions and retractions from both systems and manage them within DuckDB.

We'll break this down into several components:

1. **Define the Event Structure**: We need to define what constitutes a valid event.
2. **Initialize the Database**: Set up DuckDB for storing these events.
3. **Handle Assertions**: Accept JSON-formatted assertions and insert them into DuckDB.
4. **Handle Retractions**: Accept JSON-formatted retractions and delete matching entries from DuckDB.

Step 1: Define the Event Structure

We'll use a `struct` to represent an event with fields for the unique identifier (`event_id`), the origin of the event (`origin` , either "prolog" or "clips"), the timestamp, and the payload (a JSON object).

```
use serde::{Deserialize, Serialize};
use std::time::{SystemTime, UNIX_EPOCH};
use uuid::Uuid;

#[derive(Debug, Serialize, Deserialize)]
struct Event {
    event_id: Uuid,
    origin: String,
    timestamp: SystemTime,
    payload: serde_json::Value,
}
```

Step 2: Initialize the Database

We'll create a function to initialize DuckDB and set up our `events` table.

```
use duckdb_sys::{DuckDBResult};
use std::ffi::CString;

fn init_duckdb() -> DuckDBResult {
    unsafe { DuckDBResult(duckdb_open(CString::new("my_db").unwrap().as_ptr())) }
}

fn create_events_table(db: &DuckDBResult) {
    let query = "
        CREATE TABLE IF NOT EXISTS events (
            event_id UUID PRIMARY KEY,
```

```
            origin TEXT NOT NULL, -- 'prolog' or 'clips'
            timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
            payload JSONB NOT NULL
        );
    ";
    unsafe { duckdb_execute(db.as_mut(), CString::new(query).unwrap().as_ptr()) };
}
```

Step 3: Handle Assertions

We'll define a function to handle incoming assertions. This will parse the incoming JSON, insert it into DuckDB, and return an acknowledgment.

```
use serde_json::{json, Value};
use std::str::FromStr;

fn on_assert(db: &DuckDBResult, json_str: String) -> Result<(), Box<dyn std::error::Error>> {
    let event: Event = serde_json::from_str(&json_str)?;

    // Generate a unique ID if not provided
    let event_id = match event.event_id {
        Uuid::nil() => Uuid::new_v4(),
        id => id,
    };

    let query = format!(
        "INSERT INTO events (event_id, origin, timestamp, payload) VALUES ('{}', '{}', {}, {});",
        event_id,
        event.origin.to_lowercase(),
        event.timestamp.duration_since(UNIX_EPOCH).unwrap().as_secs(),
        event.payload.to_string()
    );

    unsafe { duckdb_execute(db.as_mut(), CString::new(query).unwrap().as_ptr()) }?;
    Ok(())
}
```

Step 4: Handle Retractions

We'll define a function to handle retractions. This will parse the incoming JSON, delete the corresponding event from DuckDB, and return an acknowledgment.

```
use serde_json::{json, Value};
use std::str::FromStr;

fn on_retract(db: &DuckDBResult, json_str: String) -> Result<(), Box<dyn std::error::Error>> {
    let retraction: Value = serde_json::from_str(&json_str)?;

    // Extract the event ID from the JSON payload
```

```
    if let Some(event_id) = retraction.get("event_id").and_then(|v| v.as_str()).map(Uuid::from_str) {
        match event_id {
            Ok(uuid) => {
                let query = format!("DELETE FROM events WHERE event_id = '{}';", uuid);
                unsafe { duckdb_execute(db.as_mut(), CString::new(query).unwrap().as_ptr()) }?;
            }
            Err(_) => eprintln!("Invalid UUID format in retraction"),
        }
    } else {
        eprintln!("Event ID not found in retraction");
    }

    Ok(())
}
```

Putting It All Together

Now, let's put all the pieces together into a single executable Rust program that can handle assertions and retractions from SWI Prolog or CLIPS.

```
use duckdb_sys::{DuckDBResult};
use std::ffi::CString;
use std::sync::{Arc, Mutex};

fn main() {
    // Initialize DuckDB
    let db = init_duckdb();
    create_events_table(&db);

    // Wrap the database in a mutex for thread-safe access
    let db_mutex = Arc::new(Mutex::new(db));

    // Example JSON strings (these would come from SWI Prolog or CLIPS)
    let assertion_json = r#"{"event_id": "00000000-0000-0000-0000-000000000001", "origin": "prolog", "payload":
{"key": "value"}}"#;
    let retraction_json = r#"{"event_id": "00000000-0000-0000-0000-000000000001"}"#;

    // Handle assertion
    if on_assert(db_mutex.lock().unwrap().as_mut(), String::from(assertion_json)).is_ok() {
        println!("Assertion handled successfully");
    }

    // Handle retraction
    if on_retract(db_mutex.lock().unwrap().as_mut(), String::from(retraction_json)).is_ok() {
        println!("Retraction handled successfully");
    }
}
```

Dependencies

Ensure you have the following dependencies in your `Cargo.toml`:

```
[dependencies]
duckdb-sys = "0.1"
serde = { version = "1.0", features = ["derive"] }
serde_json = { version = "1.0", features = ["alloc"] }
uuid = "0.8"
std-sync-mutex = "0.2"  # This is part of Rust's standard library
```

Summary

This stub implementation provides a basic API for handling JSON-formatted events from SWI Prolog and CLIPS, storing them in DuckDB with appropriate transactions and concurrency control. You can extend this further by adding more sophisticated error handling, logging, and possibly a proper HTTP server to expose the API over a network.